URBAN FEASIBILITY ENGINE · PERAK

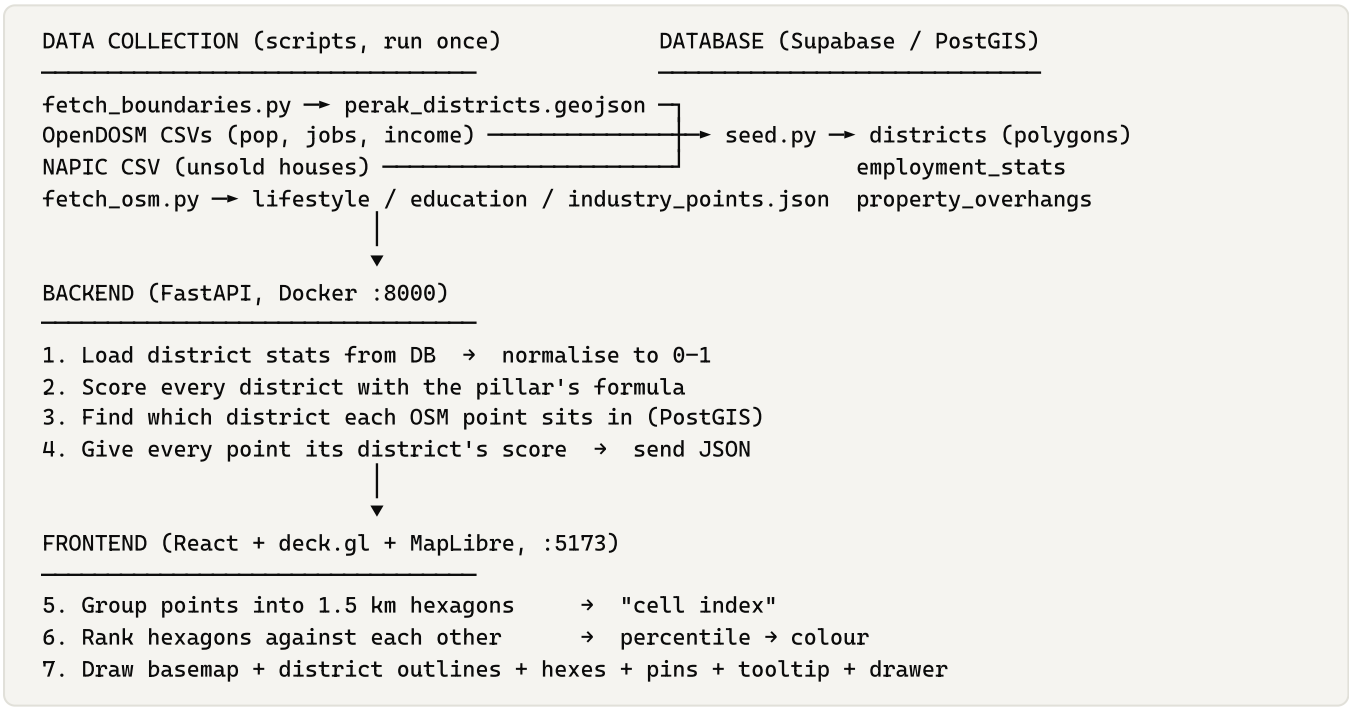
Prospectif — How It Works

A technical walkthrough of the data pipeline, the scoring brain, and the map visualisation: how district scores are calculated, how anchors become hexagon cells, and how the best micro-cities are ranked and drawn on the Perak map.

Version: October 2026 · Stack: FastAPI · PostGIS (Supabase) · React · deck.gl · MapLibre

1. The big picture
2. What each file does
3. District fundamentals
4. How the district score is calculated
5. How each point learns its district
6. Hexagons, anchors & cell index
7. How the map is drawn
8. Tooltip, drawer & verdict
9. Known limitations

1. The big picture



> **The key idea:** scores are calculated *per district* (backend) but shown *per hexagon* (frontend). A hexagon glows red when it sits in a **good district** *and* contains **many anchors** packed together.

2. What each file does

Data collection — `backend/scripts/` (run manually, once)

FILE	ROLE
`fetch_osm.py`	Queries the OpenStreetMap Overpass API for every cafe/mall, university/college and industrial zone inside Perak. Saves each as `{name, lat, lon}` in three JSON files. These are the **anchors**.
`fetch_boundaries.py`	Asks OSM Nominatim for the real outline polygon of each of the 12 districts. It accepts only district-level administrative boundaries (place_rank 12) lying inside Perak's bounding box, tries alternative names (e.g. "Bagan Datoh"), and saves `perak_districts.geojson`.
`seed.py`	Loads the database: district polygons (plus a GiST spatial index), OpenDOSM statistics and NAPIC overhang figures. Safe to re-run.

Data files — `backend/data/`

FILE	CONTENTS
`population_district.csv` `lfs_district.csv` `hh_income_district.csv`	OpenDOSM data for all of Malaysia; only Perak rows are used.
`napic_overhang.csv`	Unsold residential units per Perak district, 2026.
`*_points.json`	4,007 lifestyle + 105 education + 263 industry anchors.
`perak_districts.geojson`	The 12 district polygons.

Backend application

FILE	ROLE
`main.py`	Creates the FastAPI app, enables CORS for the frontend, mounts the routers.
`database/session.py`	Connects to Supabase via `SUPABASE_DATABASE_URL` from `.env`.
`database/models.py`	Table definitions: `districts`, `employment_stats`, `property_overhangs`.
`services/spatial_service.py`	Loads the three OSM JSON files into memory at start-up; holds `normalize()`.
`services/layers.py`	**The rulebook:** the four pillar formulas as data, metric labels, descriptions and the `score()` function.
`routers/predict.py`	**The brain:** `GET /api/predict/{layer}` performs steps 1–4 of the pipeline.
`routers/meta.py`	`/api/layers` (pillar list) and `/api/districts/boundaries` (simplified polygons for drawing).
`utils.py`	Old duplicate of the helpers. Not imported anywhere; safe to delete.

Frontend — `frontend/src/`

FILE	ROLE
`main.jsx`	Boots React with react-query (API fetching and caching).
`App.jsx`	Assembles the page: map, left panel, tooltip, drawer, zoom controls.
`store.js`	Shared state (zustand): theme, active pillar, glow toggle, selected hex, camera. Theme and pillar persist across reloads.
`lib/api.js`	Calls the backend (`VITE_API_URL` , or localhost in development).
`hooks/useLayerData.js`	Fetches a pillar's data and runs the hexagon binning; fetches boundaries and computes label positions.
`lib/hexbin.js`	**Turns points into hexagons and ranks them** (Section 6).
`lib/verdict.js`	Score breakdown maths and the auto-generated verdict.
`lib/format.js` , `lib/colors.js`	Number formatting ("Top 1%", "RM 5,136") and colour conversion.
`config/layers.js`	Visual configuration: icons, legend wording, verdict sentences, colour scales, basemaps, hex size, pin zoom threshold.
`components/MapView.jsx`	Draws every map layer (Section 7).
`components/ControlPanel.jsx` , `Legend.jsx`	Left floating panel: pillars, logic text, formula chips, legend, glow toggle, theme toggle.
`components/HoverTooltip.jsx` , `DetailDrawer.jsx` , `MapControls.jsx`	Hover popup, right-hand analytics drawer, zoom/reset buttons.

3. District fundamentals (`seed.py`)

For each of the 12 districts the seeder stores:

FUNDAMENTAL	SOURCE	DERIVATION
Population	OpenDOSM population	Perak, sex = both, age = overall, ethnicity = overall, latest date; × 1,000 (CSV is in thousands).
Employed workforce	OpenDOSM labour force	`lf_employed`, latest date, × 1,000. Stored in a column named `new_jobs_created`, but it is **total employed persons**.
Median household income	OpenDOSM income	`income_median` from the latest survey.
Unsold housing units	NAPIC	Unsold residential units, summed per district for 2026.

District names differ between sources, so they are reconciled: DOSM's "Larut Dan Matang" and "Selama" are **merged** into "Larut, Matang & Selama" (population and jobs summed, income averaged).

One fundamental is computed live in `predict.py`:

```
cafes & malls per 10k residents = lifestyle anchors in district ÷ population × 10,000
Kinta:  1,897 ÷ 931,000 × 10,000 = 20.4
```

4. How the district score is calculated

Step 1 — Normalise every metric to 0–1 (min-max)

```
normalised = (value − lowest district) ÷ (highest district − lowest district)
```

The best district becomes **1.0**, the worst **0.0**. Example — income: lowest is Hulu Perak (RM 3,701), highest is Manjung (RM 5,380):

```
Kinta:  (5,136 − 3,701) ÷ (5,380 − 3,701) = 0.855
```

METRIC	MEANING
`norm_jobs`	Employed workforce (Kinta 448,500 = 1.0; Hulu Perak 30,700 = 0.0)
`norm_income`	Median household income
`norm_overhang`	Unsold housing; 1.0 = the *most* unsold (Kinta, 1,420 units)
`norm_lifestyle`	Cafes & malls per 10k residents
`edu_index`	**Proxy, not real data:** 1.0 for Kampar, Muallim, Perak Tengah, Kinta (university towns); 0.4 for all others

Step 2 — Apply the pillar's weighted formula (`services/layers.py`)

"Inverted" terms use `1 − value`, meaning *lower is better* (e.g. less unsold housing is good).

PILLAR	FORMULA	ANCHORS USED
Apex Score	30% jobs + 30% income + 20% edu + 20% (1 − overhang)	all three
Talent Magnet	60% edu + 40% (1 − income)	education, lifestyle
Retail Vacuum	40% income + 30% jobs + 30% (1 − lifestyle density)	industry
Housing Gap	40% jobs + 30% income + 30% (1 − overhang)	all three

Worked example — Kinta, Apex Score

TERM	WEIGHT	VALUE	CONTRIBUTION
Employment base	0.30	1.000	0.300
Median household income	0.30	0.855	0.257
Education index (proxy)	0.20	1.000	0.200
Low unsold housing overhang	0.20	1 − 1.000 = 0.000	0.000
District score			**0.756**

Hulu Perak scores only 0.275 (lowest jobs and income). These are exactly the bars shown in the drawer's *Score breakdown*.

Step 3 — District ranking

Ranking districts is simply sorting their scores. For Apex: Kinta 0.756 → Muallim 0.606 → Perak Tengah 0.576 → Manjung 0.534 → Kampar 0.525 → … → Hulu Perak 0.275. The map does not show this list directly; it shows hexagon cells built on top of it.

5. How each point learns its district (PostGIS)

An OSM cafe only has a latitude and longitude. `resolve_districts()` sends every point to the database and asks "which district polygon contains this point?":

```
JOIN districts d ON ST_Covers(d.geom, ST_SetSRID(ST_MakePoint(lon, lat), 4326))
```

The GiST spatial index makes this fast, and the answer is cached in memory, so it runs once per server start. Each point then inherits its district's score as its **weight**:

```
{ "name": "Starbucks", "lat": 4.595, "lon": 101.089,
  "category": "lifestyle", "district": "Kinta", "weight": 0.756 }
```

Every point in Kinta therefore carries the same weight, 0.756. The API returns ~4,375 such points together with the formula and the full district breakdown.

6. Hexagons, anchors & cell index (`lib/hexbin.js`)

What are anchors?

Anchors are **real places from OpenStreetMap** — evidence that activity exists at a location:

- 🔵 **Lifestyle** — cafes and malls
- 🟠 **Education** — universities and colleges
- 🟢 **Industry** — industrial zones

Each pillar uses specific anchor types; Retail Vacuum, for example, uses only industrial zones (where the workers are).

How the hexagons are built

1. Perak is covered with an invisible honeycomb of hexagons, **1.5 km from centre to corner**.
2. Each point is converted from latitude/longitude to metres (a local flat projection, accurate to under 1% at Perak's latitude) and dropped into the hexagon it falls in, using axial hex-grid coordinates with cube rounding.
3. Only hexagons containing at least one anchor exist — empty jungle shows nothing. For Apex that gives 272 cells.

What is the cell index?

```
cell index = sum of the weights of every anchor inside the hexagon
           = (number of anchors) × (district score)     ← when all fall in one district

Downtown Ipoh cell:  527 anchors × 0.756 = 398.41
Hulu Perak cell:       2 anchors × 0.275 =   0.55
```

> The cell index rewards **both** a strong district **and** a high concentration of real activity. That combination is what the dashboard calls a *micro-city*.

How cells are ranked and coloured

Raw cell indexes span 0.3 to 398, so colouring them directly would paint only Ipoh red and everything else identical blue. Instead each cell gets a **percentile rank** — the share of all cells it beats (ties share a mid-rank). The percentile is split into seven colour steps:



0–14% · low Perak median 86–100% · high

PERCENTILE	VERDICT LABEL
≥ 85%	Prime opportunity
≥ 60%	Strong opportunity
≥ 40%	Moderate opportunity
< 40%	Weak opportunity

> **"Which micro-city is best?"** — the deep-red hexagons: the top ~14% of all 1.5 km cells in Perak for the selected pillar.

7. How the map is drawn (`MapView.jsx`)

The map is a stack of transparent layers rendered on the GPU by deck.gl, bottom to top:

#	LAYER	WHAT YOU SEE
1	**Basemap** (MapLibre + CARTO)	Roads, towns, sea. Positron in light mode, Dark Matter in dark mode. A background picture only — it knows nothing about the data.
2	**District boundaries** (`GeoJsonLayer`)	The 12 outlines, served from PostGIS and simplified with `ST_SimplifyPreserveTopology` (~100 m tolerance) down to 73 KB.
3	**District names** (`TextLayer`)	Placed at the centre of each district's main polygon; hidden past zoom 10.5.
4	**Glow heatmap** (toggle)	Blurred density cloud of the same points, for presentations.
5	**Hexagons** (`PolygonLayer`)	The coloured cells; hoverable and clickable. In glow mode they become invisible but stay clickable.
6	**Selected hexagon**	Thick outline around the clicked cell.
7	**Anchor pins** (`ScatterplotLayer`)	Real places, coloured by category.

Zoom handover

Between zoom 11 and 12.5 the hexagons fade to 15% strength while the anchor pins fade in. Zoomed out, the map answers "where is the opportunity?"; zoomed in, it answers "these are the actual cafes, colleges and factories that prove it".

8. Tooltip, drawer & verdict

Hover a hexagon

Shows the district, rank ("Top 1%"), cell index, district score, anchor counts and the formula's metric values.

Click a hexagon — the analytics drawer

- **Verdict** — rule-based, not AI: rank band + strongest factor + weakest factor + a pillar-specific recommendation. Example: "*Its strongest driver is employment base (1.00); the main drag is low unsold housing overhang (0.00). Prioritise for flagship mixed-use or township development.*"
- **Score breakdown** — one bar per formula term. The grey track is the term's weight; the purple fill is how much of it the district earns.

- **Anchors in this cell** — count per category and up to six named places.
- **District fundamentals** — population, workforce, income, unsold units, cafes per 10k.
- **Zoom to anchors** — flies the camera in so the pins appear.

9. Known limitations

1. **One score per district.** All places in a district share its score. Cells in Ipoh beat cells in Batu Gajah only because Ipoh has *more anchors*, not better local economics. Finer differences need sub-district (mukim) data.
2. **Cafe density dominates.** 4,007 lifestyle anchors versus 105 education and 263 industry: for Apex and Housing, "red" largely means "many cafes in a good district".
3. `edu_index` **is a placeholder** (hardcoded 1.0 / 0.4), not measured data.
4. **"Jobs" means total employed persons**, not new jobs created, despite the column name.
5. **Rankings are relative.** "Top 1%" means best within Perak for that pillar, not good in absolute terms.